

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Pruebas de Software				
TÍTULO DE LA PRÁCTICA:	Pruebas de Integración: API Testing con Postman y Supertest				
NÚMERO DE LA PRÁCTICA:	08	AÑO LECTIVO:	2026	NRO. SEMESTRE:	A
FECHA DE PRESENTACIÓN:	01/07/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(S): - Barrios Medina, Mathias Alonso - Cuno Salazar, Eduardo Joel - Hanco Mullisaca, Sergio Danilo - Sucle Suca, Michael Benjamin				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Profe. Robert Arisaca					

RESULTADOS Y PRUEBAS
Ejercicio 1: Automatización con Supertest Elijan una temática de su preferencia (ej. catálogo de videojuegos, gestión de una biblioteca musical, reserva de películas, etc.) e implementen una API REST básica en Node.js + Express. Posteriormente, diseñen y ejecuten una suite completa de pruebas de integración utilizando Supertest y Jest/Mocha. Requerimientos de la Prueba: - Flujo de Persistencia Cruzada: Validar la integración secuencial entre la creación de un recurso (POST /recurso) y su posterior lectura (GET /recurso/:id), asegurando que el ID generado dinámicamente en el primer endpoint sea el puente de entrada para el segundo. - Simulación de Modificación de Estado: Implementar y verificar un endpoint que altere una propiedad cuantitativa del recurso (ej. restar stock, cambiar nivel, actualizar reproducciones) y confirmar mediante un GET posterior que el cambio se consolidó en el mock de persistencia o base de datos de pruebas. - Validación de Robustez (Edge Cases): Asegurar que el sistema maneje correctamente los desajustes de datos o tipos inválidos (ej. enviar un texto en un campo numérico o dejar campos obligatorios vacíos) devolviendo exactamente el código de estado HTTP 400 (Bad Request) junto con su respectivo mensaje de error. Entregables: - Archivo de la API (app.js). - Archivo de pruebas (tema_libre.test.js). - Captura de pantalla o reporte en texto del resultado de la ejecución en la terminal usando Jest/Mocha. Desarrollo del Backend - Catálogo de Videojuegos I. Configuración del Proyecto Se creó el proyecto <i>videojuegos-api</i> con Node.js y Express.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```

{} package.json U X JS app.js U
C: > Programador > UNSA > Lab_Pruebas_de_Software >
1  {
2    "name": "videojuegos-api",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "jest"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC",
12   "type": "commonjs",
13   "devDependencies": {
14     "jest": "^30.4.2",
15     "supertest": "^7.2.2"
16   },
17   "dependencies": {
18     "express": "^5.2.1"
19   }
20 }

```

Figura 1: Archivo *package.json* con las dependencias instaladas

II. Servidor Implementado

Se implementaron los siguientes endpoints en *app.js*:

- POST /videojuegos - Crear videojuego
- GET /videojuegos/:id - Leer videojuego
- PUT /videojuegos/:id/stock - Actualizar stock

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

- DELETE /videojuegos/:id - Eliminar videojuego
- DELETE /videojuegos - Limpiar todos (para pruebas)

Validaciones en POST:

- Campos obligatorios: nombre, genero, stock, precio
- Tipos de datos: stock y precio numéricos
- Valores lógicos: stock y precio ≥ 0

III. Servidor en Ejecución

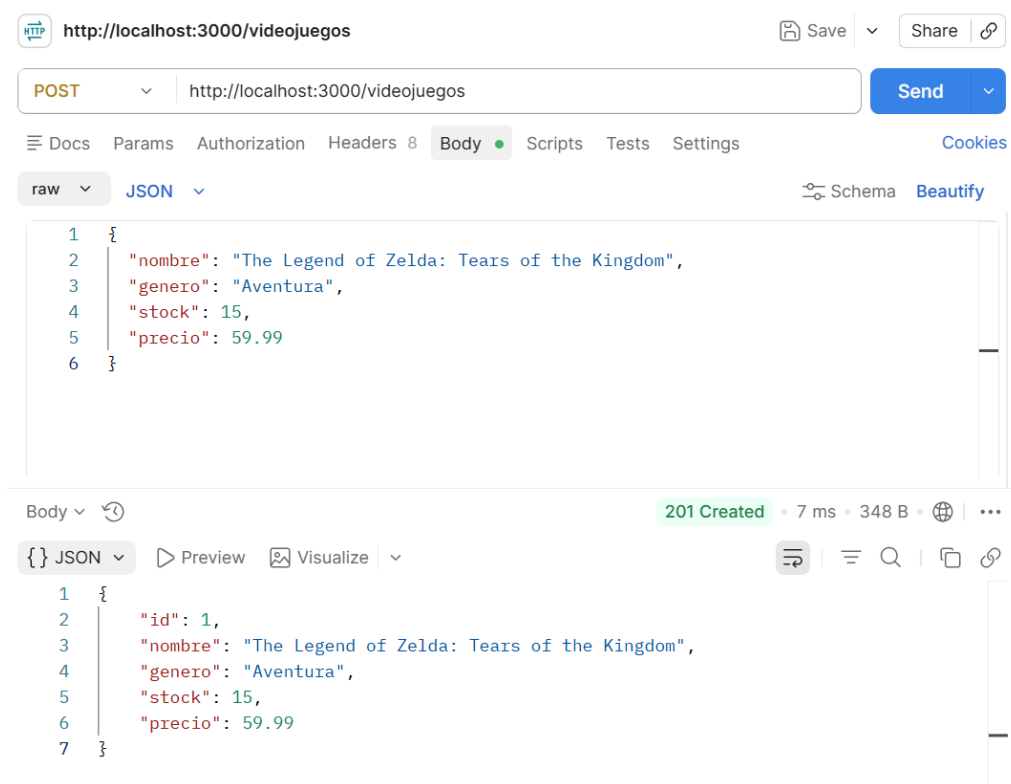
El servidor corre en `http://localhost:3000`.

```
C:\Programador\UNSA\Lab_Pruebas_de_Software\Laboratorio_8\videojuegos-api>node server.js
=====
SERVIDOR DE VIDEOJUEGOS
=====
Servidor corriendo en:
http://localhost:3000
Endpoints disponibles:
POST /videojuegos - Crear videojuego
GET /videojuegos/:id - Leer videojuego
PUT /videojuegos/:id/stock - Actualizar stock
DELETE /videojuegos/:id - Eliminar videojuego
DELETE /videojuegos - Limpiar todos (para pruebas)
=====
Presiona Ctrl+C para detener el servidor
```

Figura 2: Servidor ejecutándose en la terminal

IV. Pruebas Manuales con Postman

4.1. Crear Videojuego - POST



The screenshot shows a Postman interface for a POST request to `http://localhost:3000/videojuegos`. The request body is a JSON object:

```
{
  "nombre": "The Legend of Zelda: Tears of the Kingdom",
  "genero": "Aventura",
  "stock": 15,
  "precio": 59.99
}
```

The response status is **201 Created**, and the response body is a JSON object:

```
{
  "id": 1,
  "nombre": "The Legend of Zelda: Tears of the Kingdom",
  "genero": "Aventura",
  "stock": 15,
  "precio": 59.99
}
```

Figura 3: Resultado: Status 201, ID asignado: 1

4.2. Leer Videojuego - GET

My Collection > Get data Save Share

GET Send http://localhost:3000/videojuegos/1

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK 6 ms 343 B Save Response

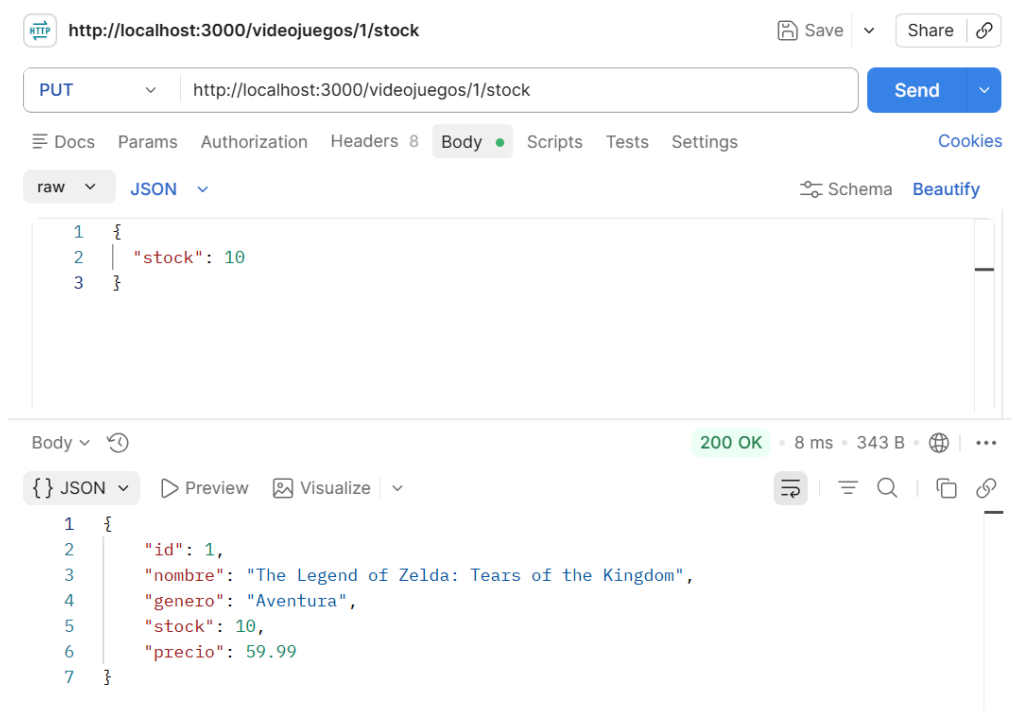
{} JSON Preview Visualize

```

1  {
2    "id": 1,
3    "nombre": "The Legend of Zelda: Tears of the Kingdom",
4    "genero": "Aventura",
5    "stock": 15,
6    "precio": 59.99
7  }
```

Figura 4: Resultado: Status 200, datos correctos

4.3. Actualizar Stock - PUT



The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/videojuegos/1/stock`
- Method:** PUT
- Body (JSON):**

```

1 {
2   "stock": 10
3 }

```
- Status:** 200 OK (8 ms, 343 B)
- Response Body (JSON):**

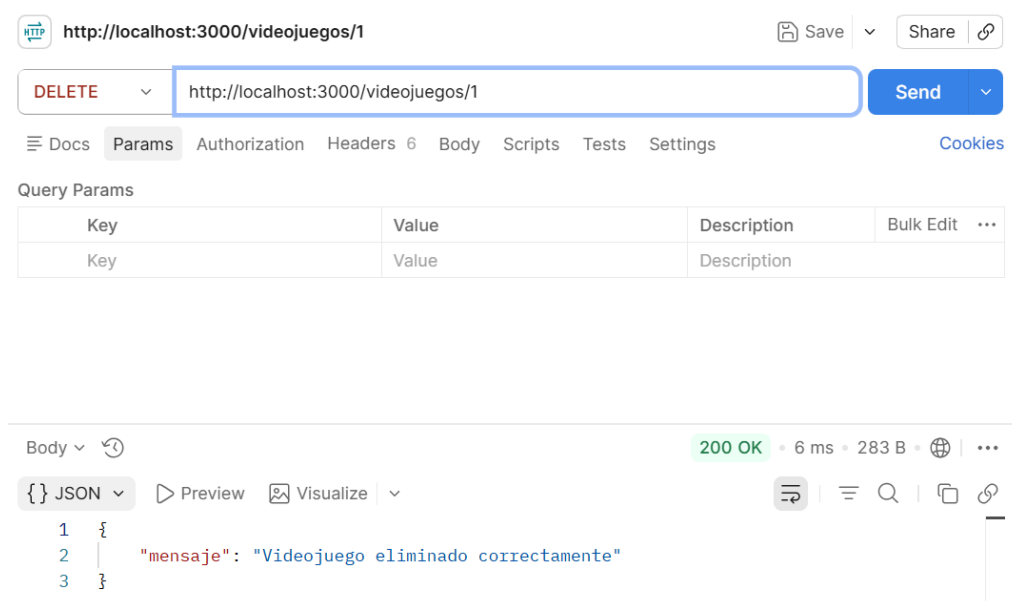
```

1 {
2   "id": 1,
3   "nombre": "The Legend of Zelda: Tears of the Kingdom",
4   "genero": "Aventura",
5   "stock": 10,
6   "precio": 59.99
7 }

```

Figura 5: Resultado: Status 200, stock actualizado

4.4. Eliminar Videojuego - DELETE



The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/videojuegos/1`
- Method:** DELETE
- Query Params Table:**

Key	Value	Description	Bulk Edit	...
Key	Value	Description		
- Status:** 200 OK (6 ms, 283 B)
- Response Body (JSON):**

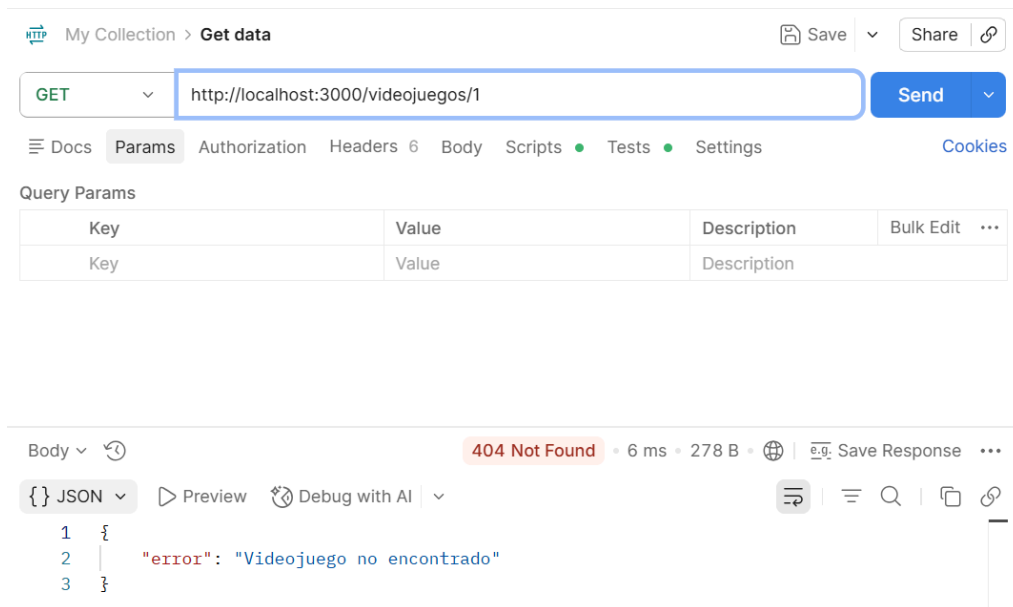
```

1 {
2   "mensaje": "Videojuego eliminado correctamente"
3 }

```

Figura 6: Resultado: Status 200, mensaje de éxito

4.5. Verificar Recurso Eliminado - GET 404



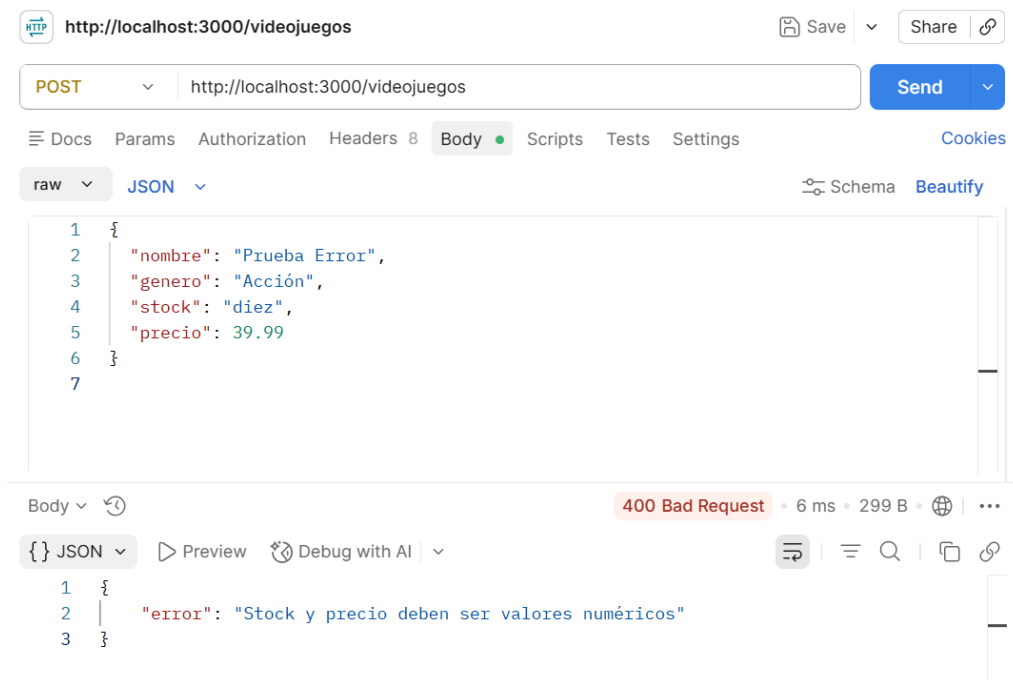
The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:3000/videojuegos/1
- Status:** 404 Not Found
- Response Body (JSON):**

```
{
  "error": "Videojuego no encontrado"
}
```

Figura 7: Resultado: Status 404, "Videojuego no encontrado"

4.6. Validaciones - Errores 400



The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3000/videojuegos
- Status:** 400 Bad Request
- Request Body (JSON):**

```
{
  "nombre": "Prueba Error",
  "genero": "Acción",
  "stock": "diez",
  "precio": 39.99
}
```
- Response Body (JSON):**

```
{
  "error": "Stock y precio deben ser valores numéricos"
}
```

Figura 8: Resultado: Status 400, "Stock y precio deben ser valores numéricos"

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

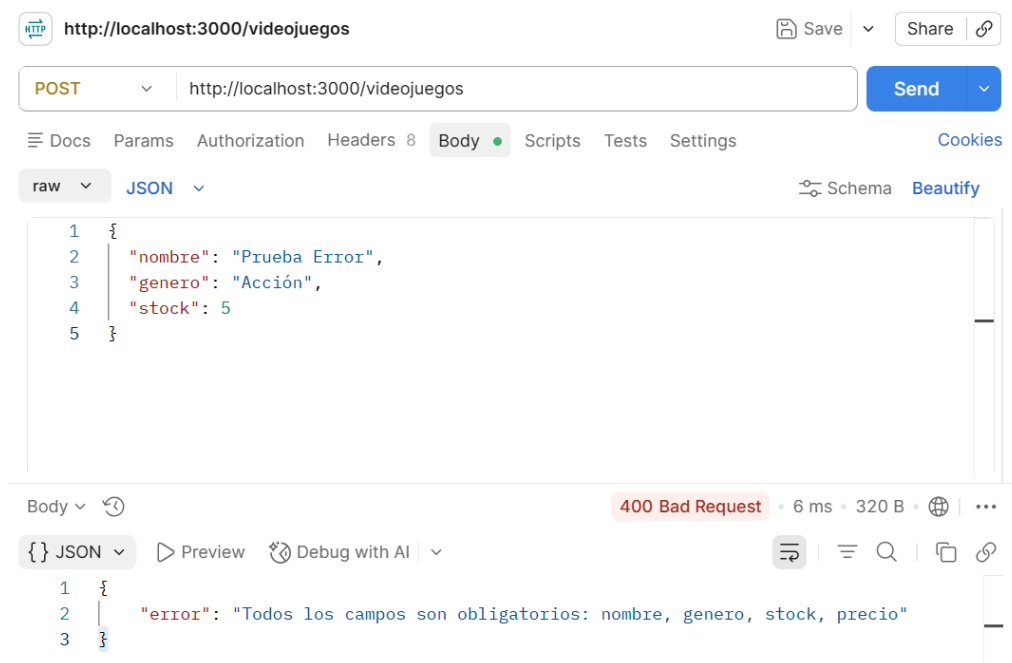


Figura 9: Resultado: Status 400, "Todos los campos son obligatorios"

Obteniendo nuestro sistema listo para ser testeado

- Servidor implementado: Completado
- Servidor en ejecución: Completado
- Pruebas manuales Postman: Completado
- Capturas de evidencias: Completado

Ejercicio 2: Pruebas de Integración del Proyecto Final

El grupo debe aplicar pruebas de integración sobre la arquitectura de su proyecto de fin de curso. El enfoque debe centrarse en verificar la cohesión y el flujo de datos entre los componentes que han sido desarrollados. El grupo puede elegir la herramienta (Requests, Supertest, REST Assured, Playwright, etc.) que mejor se adapte a su lenguaje de programación.

Tarea de Análisis de Errores:

1. **Mapeo de la Frontera:** Identifique el punto donde el Subsistema A entrega el control o los datos al Subsistema B.
2. **Inyección de Fallas de Interfaz:** Diseñe al menos 3 casos de prueba orientados a "romper" la comunicación:
 - **Caso 1 (Sintáctico):** Envíe un objeto con campos faltantes o tipos de datos erróneos.
 - **Caso 2 (Semántico):** Envíe valores legales pero fuera de lógica para el receptor (ej. una fecha de entrega anterior a la de pedido).
 - **Caso 3 (Resiliencia):** Simule una latencia alta en la respuesta del subsistema B y analice si el subsistema A maneja el timeout o colapsa.
3. **Documentación de Discrepancias:** Por cada falla encontrada, complete un Reporte de Incidente que detalle el "Resultado Esperado" vs. el "Resultado Real" observado en la interfaz.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

CUESTIONARIO

Pregunta 1: ¿Por qué las pruebas unitarias exitosas no garantizan que la integración será exitosa?

Porque una prueba unitaria verifica un módulo de forma aislada, reemplazando sus dependencias con dobles de prueba [1]. Esto significa que nunca se prueba la comunicación real entre módulos. Aunque cada módulo funcione perfectamente por separado, al integrarlos pueden surgir problemas como: diferencias en los formatos de datos que espera cada módulo, suposiciones contradictorias sobre el estado del sistema, errores en los contratos de las interfaces (nombres de métodos, tipos de parámetros, orden de llamadas), o defectos enmascarados donde un error en un módulo es ocultado temporalmente por otro. Las pruebas de integración son las únicas que pueden detectar estos fallos en las interacciones entre componentes reales.

Pregunta 2: Explique la diferencia entre un Stub y un Driver y proporcione un ejemplo de cuándo usó uno de ellos en su proyecto.

Pregunta 3: Según Myers, ¿por qué es arriesgado que el mismo desarrollador que escribió el código de los módulos diseñe también las pruebas de integración?

Myers [2] sostiene que el desarrollador que escribió el código tiene **sesgos cognitivos** naturales: conoce su propia lógica y tiende a probar los caminos que ya sabe que funcionan, pasando por alto escenarios que no consideró al programar. En integración, estos sesgos son particularmente peligrosos porque las fallas suelen estar en los **límites** entre módulos, no dentro de ellos. El desarrollador asume que su interfaz se comunica correctamente porque así lo diseñó, pero un ojo fresco —que no tiene esas suposiciones grabadas— tiene más probabilidades de encontrar los defectos reales en la comunicación entre componentes. Myers recomienda que un tercero diseñe las pruebas de integración para romper ese sesgo y garantizar una cobertura más objetiva.

Pregunta 4: Defina “Integración Incremental” y explique por qué el enfoque “Big Bang” debe evitarse en proyectos complejos.

La **Integración Incremental** consiste en integrar y probar los módulos de uno en uno (o en pequeños grupos), añadiendo módulos gradualmente y probando después cada adición. Esto permite aislar defectos rápidamente: si al agregar un módulo aparece un error, el problema está en la interacción entre el nuevo módulo y los ya integrados.

El enfoque **Big Bang**, por el contrario, integra todos los módulos de una sola vez al final del desarrollo. En proyectos complejos esto es peligroso porque, cuando ocurre una falla, es imposible determinar qué interacción entre qué módulos la causó. Hay demasiadas variables simultáneas: decenas o cientos de interfaces comunicándose por primera vez. Depurar se convierte en encontrar una aguja en un pajar. El Big Bang retrasa la detección de errores hasta el final del ciclo, cuando corregirlos es más caro y riesgoso.

Pregunta 5: ¿Cómo ayuda la herramienta seleccionada por su grupo a detectar “defectos enmascarados” entre sus subsistemas?

Las herramientas que seleccionamos **Postman** y **Supertest** ayudan a detectar defectos enmascarados al validar las **interacciones reales** entre subsistemas a través de sus APIs HTTP. Un defecto enmascarado ocurre cuando un error en un subsistema queda oculto porque otro subsistema compensa temporalmente ese error (por ejemplo, con valores por defecto o conversiones implícitas). Postman permite crear colecciones de pruebas que encadenan varias llamadas API, simulando flujos completos del sistema; si un subsistema devuelve datos inesperados pero el siguiente los procesa sin fallar, las aserciones de Postman pueden detectar que la respuesta final no coincide con el contrato esperado. Supertest, por su parte, permite escribir pruebas automatizadas con aserciones precisas sobre los códigos de estado, el cuerpo y los encabezados de cada respuesta. Así, si un cambio en un subsistema modifica sutilmente su salida y otro subsistema se adapta silenciosamente, la prueba falla y revela el defecto enmascarado que de otro modo pasaría desapercibido en producción.

CONCLUSIONES

Conclusión 1

Contenido de la conclusión 1...

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

REFERENCIAS
Bibliography [1] A. Spillner, <i>Software Testing Foundations</i>, 5th ed. Rocky Nook, 2021. [2] G. Myers, <i>The Art of Software Testing</i>, 3rd ed. Wiley, 2012.